PsySuite 2 Developer Guide

Introduction

This guide will help developers to create new tasks in the PsySuite framework. It will introduce the core classes and methods that the framework (module *psysuitecore*) exposes to the developers that want to add a new Test in *psysuitetests* module. The framework will be described by showing how to implement two tasks: a simple reaction time and a task following the 2AFC paradigm. New Tasks must be created in psysuitetests library module. Psysuitecore code should not be modified unless, while creating a new task, the developer needs new features that can be reused by other tests. In that special case, those new common features should go into psysuitecore. If the new task includes an adaptive modality that necessitates a specific algorithm that must communicate with the python engine (to access ADOpy primitives or any other python package), such python code must be added into the psysuitepython library module. The Nativeaudio library module should never be modified, playback primitives are already present and accessed by Kotlin wrappers defined in psysuitecore module.
With *Test* we will refer to the implementation of a specific task, with *experiment* we will refer to a subject that runs a Test, thus the latter information will include demographic data, experiment date, selected condition, device specification and so on.
The code included in this guide is a simplified version and thus differ from the actual implementation present in the repositories.

Preliminary information

To create a new Test, user must implement three different classes, extending the following classes defined in psysuitecore:

- BasicSettings
- TrialBasic
- TestBasic

The first contains experiments data (e.g. subject demographic) and Test settings that can be selected and modified before Task start, `TrialBasic` contains trial parameters (e.g. progressive number, experimental condition, stimulus magnitude and user response) and the last is the main class containing the Test logic (trials composition, stimuli delivery, etc). These classes operate together with other two classes: `SubjectBasicDialogFragment`, a dialog that lets user select experiment and Task data, whom output is an instance of `BasicSettings`, and, when applicable, a dialog fragment to collect user answer at the end of each trial. If a Task requires additional settings not included in `SubjectBasicDialogFragment` or user responses are more complex than 2 or 3 alternative responses (already implemented in

`TwoAFCAnswerDialogFragment` and `ThreeAFCAnswerDialogFragment`), developer must also implement its own version of these dialog fragments.

Let's see in more details, the three main classes that must be extended to implement a new Test.

BasicSettings

This class contains Test parameters and settings that define its behavior, for example what to do when a trial ends, and experiments data (e.g. participant info, experiment date, etc.).

This is the list of the parameters defined in *BasicSettings* each corresponding to a specific graphical element in the *SubjectBasicDialogFragment* (or its Test's subclass when needed)

Unless differently specified, most *BasicSettings* properties have four possible values:

- TEST_SWITCH_DISABLED: a test feature/switch is disabled and cannot be enabled, thus is invisible
- TEST_SWITCH_CHOOSE_OFF: a test feature/switch can be chosen by the user and is off by default
- TEST_SWITCH_CHOOSE_ON: a test feature/switch can be chosen by the user and is on by default.
- TEST_SWITCH_ENABLED: a test feature/switch is enabled and cannot be disabled, thus is invisible

Parameters regulated by these values are marked with (*****)

Information on the subject participating in the given experiment:

Label (free text), *age* (only number text), *gender* (radio button),
Population: used to define whether subject belongs to a clinical population or has some sensory impairment (e.g. blind or deaf). See `org.albaspazio.psysuite.core.models.Populations` for the list of available populations.

Test information

`class`: the class namespace of the custom Test instance
`settingsclass`: if necessary (default to empty string), is the class namespace of the custom SubjectDialogFragment instance where user insert/select experiment data
`answerclass`: if necessary (default to empty string), is the class namespace of the custom answer DialogFragment instance where user insert its response at the end of each trial.
`type`: the unique id of the Test condition selected by the user (e.g. whether auditory or visual)

Experiment information

`project`: the project the experiments belong to.

`session_spsel:`
`session_spdatares`

Information automatically filled by the system

`block`: indicates whether is a part/block of an experiment
`exp_uid`: unique code to distinguish experiments when uploaded on the results server
`date`: date when experiment was completed

`device`: json parameter describing device os, name, manufacturer, model, id, total/free memory
`vercode`: psysuite version code

Test settings:

These parameters define the behavior of the Test, mainly regarding what to do when a trial ends.

nextTrialModality**:** what to do when a trial end:

- TEST_NEXTTRIAL_NOCHOOSE: proceeds directly to the next trial without user choice.
- TEST_NEXTTRIAL_AUTO: user can only choose to abort/pause the test after each trial end
- TEST_NEXTTRIAL_BUTTON: user can choose to wait and then press a 'NEXT' button
- TEST_NEXTTRIAL_ANSWER: test waits for an answer dialog.
- TEST_NEXTTRIAL_VOICE_ANSWER: test waits for a voice answer dialog via speech recognition
- TEST_NEXTTRIAL_VOICE_NORMAL_ANSWER: test waits for either a normal answer dialog or a voice answer.

`whitenoise` (*): playback or not a white noise during each trial. E.g. when I want to cover the sound produced by the vibrators engine during tactile stimulation

`trman_type`: deliver fixed or adaptive trials

- TEST_TRMAN_FIXED: trials are predetermined at the start of the test, cannot be modified
- TEST_TRMAN_CHOOSE_FIXED: user can choose, with predetermined trials as the default
- TEST_TRMAN_CHOOSE_ADAPTIVE: user can choose, with adaptive trials as the default
- TEST_TRMAN_ADAPTIVE: all trials are adaptive, cannot be modified

`showResults` (*): show results at the end of each trial
`canRepeat` (*): give the possibility to repeat a trial
`doTraining` (*): do a training session before task start

showTrialId: show a number indicating the current trial number:

- TEST_SHOWTRIALS_NEVER: Never
- TEST_SHOWTRIALS_TRIALEND: At the end of the trial:
- TEST_SHOWTRIALS_ALWAYS: Always

abortMode: whether show the abort button:

- TEST_ABORT_TRIALEND: only at the end of the trial
- TEST_ABORT_ALWAYS: also during the trial

In addition, this class manage the names of the results file (a json with BasicSettings content and a txt file with experiment results) and writes the json file.

TrialBasic

This class gets initialized with the following properties

```
var id: Int = -1,                // progressive number of trial
val type: Int,                   // condition code
protected val label: String = "", // label of the condition
open var magnitude: Float = 0F,  // stimulus magnitude
val adoWrapper: ADOWrapper? = null,     // reference to the Adaptive wrapper
                                        // null in case of a trial with fixed magnitude
val isTraining: Boolean = false  // defined whether is a training trial
```

And has the following properties

```
companion object {
    @JvmStatic val LOG_HEADER = "id\tlabel\tres\tcor_ans\tuser_ans\telapsed\trep\n"
}
```

that defines the number and label of the results file

```
var user_answer:Int             = -1    // store user response
var repetitions:Int             =  1
var elapsed:Long                = -1L
var prev_trial: TrialBasic?     = null  //in case more complex processing is needed
var user_answer_extra:String    = ""
open var correct_answer:Int     = 0
var success:Boolean             = false // result of comparison between correct and
                                        // user answer
```

and has the following methods

```
initTrial(newvalue:Float)
```

during Adaptive trials, set the magnitude of the next trial (as obtained from adaptive engine)

```
setResponse(result: Int, elapsedms: Long = -1L, prev_tr: TrialBasic? = null,
extra_text:String="")
```

called at the end of the trial, set the subject response

```
getAdoUpdatingMethod():String
```

return the name of the update method of the Adaptve engine. By default, is “set”.

```
getAdoUpdatingParams():List<Any>
```

return the Trial properties (response, success, etc..) to pass to Adaptive engine. By default, is the response and stimulus magnitude

```
val stim_value:Long
```

This property returns the stimulus intensity in the Test spatial and/or temporal coordinates, by default coincides with the magnitude, but may be an its manipulation.

```
open fun Log():String{
  return "$id\t$label\t$success\t$correct_answer\t$user_answer\t$elapsed\t$repetitions\n"
}
```

defines the values of each line of the results file, return them as string

TestBasic

Is the main class of a Test. Manage all the aspects of running an experiment.

It has the following abstract methods that each subclass must implement:

- initTest()
- show(trial: TrialBasic, isRepeat:Boolean=false)

initTest initializes stimuli managers (those that deliver stimuli), trials manager (defining the number and characteristic of the trials to execute) and run several other initializations.

Show is the method that run a single trial, finally calls `onStimuliEnd`().

In its companion object

Extra classes

If Task needs extra configuration parameters, developers should implement a new class extending `SubjectBasicDialogFragment` and indicate its classname, as string, in `settingsclass` parameter of `SettingsBasic` derived class.

If Task needs special responses (like TestTFI), developers should implement a new class extending `DialogFragment`, and indicate its classname, as string, in `answerclass` parameter of `SettingsBasic` derived class

Reaction times

This is the simplest test, a stimulus is delivered, and user must press a button on device screen as fast as possible. The App will record the time elapsed between stimulus delivery and subject button press in a proper text file. Target stimuli can be delivered in three different sensory modalities, each creating a different sub-task/condition with its own unique code. Each time you want to create a new test, you must define a unique ID for each test condition. Test codes must be inserted into the companion object of `org.albaspazio.psysuite.tests,TestBasic` class, contained in `psysuitecore` library module.

1) Create the package

Create the test package in `org.albaspazio.psysuite.tests.RT`. This folder will contain all the classes implementing the test.

2) Create the parcel class

Create the parcel class, extending the `SettingsBasic` class, that stores the test parameters, name it `SettingsRT`.

```
class SettingsRT(

    override var classes: List<String> =
               listOf("org.albaspazio.psysuite.tests.rt.TestRT"),
    override var label: String = "",
    override var age: Int = -1,
    override var gender: Int = -1,
    override var population: Int = Populations.POPULATION_TD,
    override var type: Int = -1,
    override var project: String = "",

    override var block: Int = -1,
    override var isDebug: Boolean = false,
    override var device: Device? = null,
    override var vercode: Int = -1,
    override var stimuliDelays: DelaysAligner = DelaysAligner(),

    override var nextTrailModality: Int = TestBasic.TEST_NEXTTRIAL_AUTO,
    override var whitenoise: Int = TestBasic.TEST_SWITCH_DISABLED,
    override var trman_type: Int = TestBasic.TEST_TRMAN_FIXED,
    override var showResult: Int = TestBasic.TEST_SWITCH_DISABLED,
    override var canRepeat:Int = TestBasic.TEST_SWITCH_DISABLED,
    override var doTraining: Int = TestBasic.TEST_SWITCH_DISABLED,

    override var showTrialID: Int = TestBasic.TEST_SHOWTRIALS_NEVER,
    override var abortMode: Int = TestBasic.TEST_ABORT_TRIALEND,

    override var session_spsel: Int = TestBasic.Companion.TEST_LONGITUDINAL_TOBESELECTED,
    override var session_spdatares: Int = R.array.sessions_array,
    override var date: String = "",
    override var exp_uid: String = ""

): SubjectBasicParcel(classes, label, age, gender, population, type, project, block,
isDebug, device, vercode, stimuliDelays, nextTrailModality, whitenoise, trman_type,
showResult, canRepeat, doTraining, showTrialID, abortMode, session_spsel,
session_spdatares, date, exp_uid)
```

3) create test class

Create the test class: `TestRT`

make it derive from `TestBasic` and add the following parameters in the constructor

```
class TestRT(ctx: Context, activity: Activity, hostfragment: Fragment, subject:
SubjectBasicParcel, vibrator: VibrationManager?, mImageView: ImageView?,
speechManager:SpeechManager?) : TestBasic(ctx, activity, hostfragment, subject, vibrator,
mImageView, speechManager)
```

The editor warns you that four abstract methods of `TestBasic` needs to be defined.

- `InitTest`
- `onTrialEnd`
- `show`
- `initSummary`

Let the editor do it for you.

4) create Trial class

TrialRT defines the header of its output data as:

```
companion object {
    @JvmStatic val LOG_HEADER = "id\tlabel\tresponse\terror\n"
}
```

and the content of each line

```
override fun Log():String{
    return "$id\t$label\t$user_answer\t${!success}\n"
}
```

The error column (whom value is !success) is defined in

```
override fun setResponse(result:Int, elapsedms:Long, prev_tr: TrialBasic?,
extra_text:String) {
    super.setResponse(result, elapsedms, prev_tr, extra_text)
    success  = result != -1
}
```

Results is set to -1 when user does not answer within a specific interval from stimulus onset.

5) Define unique test codes

In `TestBasic` define (actually, they are already present) a unique code for each of the sub-task

```
 TEST_RT_AUDIO = 1
 TEST_RT_TACTILE = 2
 TEST_RT_VISUAL = 3
```

One for each sensorial modality.

6) Define mandatory properties:

Define the following properties of interest

create the LOG_TAG for logging functions and create the companion object to define static properties

```
override var LOG_TAG:String = TestRT::class.java.simpleName
companion object {
    TEST_BASIC_LABEL                = "RT"   // will be written in result files
   @JvmStatic var NUM_TRIALS                      = 32
    STIMULUS_DURATION_VISUAL:Long    = 50
    STIMULUS_DURATION_TACTILE:Long   = 50
    STIMULUS_DURATION_AUDIO:Long     = 50

    STIMULUS_TYPE_AUDIO              = "AUDIO"
    STIMULUS_TYPE_TACTILE            = "TACTILE"
    STIMULUS_TYPE_VISUAL             = "VISUAL"
    STIMULUS_TYPE_AUDIO_LOG          = "A"
    STIMULUS_TYPE_TACTILE_LOG        = "T"
    STIMULUS_TYPE_VISUAL_LOG         = "V"
```

then add two static methods

```
fun getConditionsInfo(ctx: Context): List<ConditionData>{
  return if(VibrationManager.sysHasVibrator(ctx))
           mutableListOf(ConditionData(TEST_BASIC_LABEL + "_" + STIMULUS_TYPE_AUDIO,
TEST_RT_AUDIO, "${TEST_BASIC_LABEL}${STIMULUS_TYPE_AUDIO_LOG}",
Populations.hearing_populations),
           ConditionData(TEST_BASIC_LABEL + "_" + STIMULUS_TYPE_TACTILE, TEST_RT_TACTILE,
"${TEST_BASIC_LABEL}${STIMULUS_TYPE_TACTILE_LOG}", Populations.all_populations),
           ConditionData(TEST_BASIC_LABEL + "_" + STIMULUS_TYPE_VISUAL, TEST_RT_VISUAL,
"${TEST_BASIC_LABEL}${STIMULUS_TYPE_VISUAL_LOG}", Populations.sighted_populations))
  else
           mutableListOf(ConditionData(TEST_BASIC_LABEL + "_" + STIMULUS_TYPE_AUDIO,
TEST_RT_AUDIO, "${TEST_BASIC_LABEL}${STIMULUS_TYPE_AUDIO_LOG}",
Populations.hearing_populations),
               ConditionData(TEST_BASIC_LABEL + "_" + STIMULUS_TYPE_VISUAL,
TEST_RT_VISUAL, "${TEST_BASIC_LABEL}${STIMULUS_TYPE_VISUAL_LOG}",
Populations.sighted_populations),
         )
   }

fun getNextTrialModes(ctx: Context):List<List<Int>>{
  return if(VibrationManager.sysHasVibrator(ctx))
            listOf( listOf(TEST_NEXTTRIAL_NOCHOOSE), listOf(TEST_NEXTTRIAL_NOCHOOSE),
     listOf(TEST_NEXTTRIAL_NOCHOOSE))
         else
            listOf( listOf(TEST_NEXTTRIAL_NOCHOOSE), listOf(TEST_NEXTTRIAL_NOCHOOSE))
   }
}
```

getConditionsInfo is the way each test class exposes the available sub-tasks to the SubjectDialogFragment. Each element of the list corresponds to an entry of its combobox Conditions

getNextTrialModes defines what should happen at the end of a trial.

In this case the TEST_NEXTTRIAL_NOCHOOSE flag says that subject does not have to do anything, and the next trial can start. Both methods accept a single parameter of type Context that has two main functions, access resources (e.g. text to display) and verify whether the device can vibrate or not (`VibrationManager.sysHasVibrator(ctx))`.

In the latter case, sub-tasks involving tactile stimuli are not displayed in the User info dialog and thus cannot be selected.

Other options are:

```
TEST_NEXTTRIAL_AUTO = 0 // user can select to go directly to next trial or abort
TEST_NEXTTRIAL_BUTTON = 1 // user can select to wait and then press a NEXT button
TEST_NEXTTRIAL_ANSWER = 2 // wait for ANSWER dialog
TEST_NEXTTRIAL_VOICE_ANSWER = 3 // wait for VOICE ANSWER dialog through speech recogn.
TEST_NEXTTRIAL_VOICE_NORMAL_ANSWER = 4 // wait for either ANSWER dialog or VOICE ANSWER
                                          through speech recogn
```

Define how to present each modality.

- Audio stimuli can be created from four different sources (tone, high latency wav, low latency with AudioTrack and very low latency with Oboe.
- Visual stimuli can be presented in two different ways, making them visible or invisible or making them switch from one image (off state) and another one (on state).
- Tactile stimuli can be single stimuli or predetermined sequences.

Visual and tactile stimuli performance differences were not investigated, and the best option (`StimuliManager.STIM_TYPE_V1 or STIM_TYPE_V2, StimuliManager.STIM_TYPE_T1 or STIM_TYPE_T2`) should be selected according to the requested scenario. Audio stimuli for psychophysics task must use Oboe method (`StimuliManager.STIM_TYPE_A4`). In case of several long audio files that do not need very fast performance can be played back with Media player API (`StimuliManager.STIM_TYPE_A2`).

```
private var STIM_A = StimuliManager.STIM_TYPE_A4
private var STIM_V = StimuliManager.STIM_TYPE_V2
private var STIM_T = StimuliManager.STIM_TYPE_T1
```

Visual stimuli must be defined

```
override var mDrawablesResource: MutableList<Int> =
mutableListOf(R.drawable.white_circle, R.drawable.blue_circle)
```

In case of white background, the off state could be the first element of such list

7) Init and set up the task

initTest

This method is automatically called by the `TestFragment` method. Users must not call it but only implement its main functionalities. In detail, they are:

- Sanity checks
- Define stimuli characteristics
- Create task trials
- Send EVENT_TEST_SETUP_COMPLETED event

If the task can use tactile stimulation, verify that vibrator property is not null. This variable is automatically calculated by the App at its startup. For example, tablets usually do not have vibrator engines and this variable is null.

```
override fun initTest(){
when {
    mImageView == null -> throw ImageViewDefinedException("IMAGE_VIEW_NOT_DEFINED")
    vibrator == null -> throw VibratorNotDefinedException("VIBRATOR_NOT_DEFINED")
}
```

Set the task label according to subject.type

```
mTestLabel = ""
getConditionsInfo(ctx).map {
   if (it.id == subject.type) mTestLabel = it.label
}
if(mTestLabel.isEmpty()) throw Exception("ERROR in TestRT.initTest: type code was not
recognized")
```

Since vibrator engines also produce a tiny sound, it is advisable to add a white noise to all those tasks involving multimodal stimulation. Users asked to attend tactile stimuli could instead implicitly attend its sound

```
mNoise = AudioManager.getAudioResource(ctx,"wnoise_20s", 0.01f)
```

If this line is omitted (or contained within an if) the mNoise property remains null and further call like mNoise?.start() does not have any effect
Define the source of each stimulus.

```
mStimuliManager = StimuliManager(
   AudioManager(STIM_A, audioResources[STIMULUS_DURATION_AUDIO] ?: "t1000hz_50ms.wav",
duration = STIMULUS_DURATION_AUDIO, ctx = ctx, handler = mStimuliHandler),
   TactileManager(vibrator!!, duration = STIMULUS_DURATION_TACTILE, handler =
mStimuliHandler, type = STIM_T),
   VisualManager(STIM_V, mImageView!!, mDrawablesResource[1], mDrawablesResource[0],
duration = STIMULUS_DURATION_VISUAL, handler = mStimuliHandler),
   delaysAligner, ctx, mStimuliHandler)
```

Call createTrials() and pass trials list to the Trial Manager

```
mTrialsManager = FixedTrialsManager(createTrials() as MutableList<TrialBasic>)
```

At the end of *initTest*, when all set up is over, the test class can send the EVENT_TEST_SETUP_COMPLETED and the task can start

```
testEvent.accept(Triple(EVENT_TEST_SETUP_COMPLETED, null, listOf()))
```

createTrials

To define trials, the *mTrails* property of TestBasic must be filled with instances of TrialBasic or any of its subclasses.
The simplest trial, an instance of TrialBasic, has the following properties:

```
open class TrialBasic(var id:Int=-1, val type:Int, protected val label:String="", var correct_answer:String="") {
```

A progressive id, a type code, a label and which is the correct answer (when applicable). In case of a RT task, all trials are of the same type and there isn't a correct answer. Success may be false only if users do not respond before.
The method createTrials, thus simply appends the mTrials list with instances of TrailBasic where only the id parameter differs, both type and label reflect the sensory modality used.

```
private fun createTrials(){
  for(i in 0 until NUM_TRIALS)
    mTrials.add(TrialRT(i, subject.type, mTestLabel))
}
```

8) Show stimuli

Once TestFragment receives the event EVENT_TEST_SETUP_COMPLETED sent by TestRT instance, it does some internal stuff and then starts the task, calling its show() method

```
override fun show(trial: TrialBasic, isRepeat: Boolean) {
   mNoise?.start()
   val stimulus_onset = (500L + random()*1000).toLong()
   mStimuliHandler.postDelayed({
       deliverStimulus(trial)
       testEvent.accept(Pair(EVENT_STIMULI_START, null))
   }, stimulus_onset)
   mStimuliHandler.postDelayed({ onStimuliEnd() }, stimulus_onset + mTrialAbortTime)
                                     // when max time is reached => force trial end
   createResponseButton("press", binding.root, ::onStimuliEnd)
}
```

Whether defined, noise starts. To reduce subjects' expectations, define a random interval (500 < t < 1500 ms) before presenting the stimulus. Set a handler to postpone stimulus presentation. Set up another handler to force trial end after `mTrialAbortTime`.

Stimulus is delivered through

```
private fun deliverStimulus(trial: TrialBasic){
    when(trial.type) {
        TEST_RT_AUDIO    ->  mStimuliManager.deliverAStimulus()
        TEST_RT_TACTILE  ->  mStimuliManager.deliverTStimulus()
        TEST_RT_VISUAL   ->  mStimuliManager.deliverVStimulus()
    }
    mTrialStartOnset = Date().time
}
```

9) End trial

A Trial can end in two ways, user acts on user interface or a dialog is opened to let user select an answer. In both cases, the custom task must call the method onStimuliEnd which is defined in TestBasic but may be overridden in the custom task class. In the reaction time tasks, we use the second approach overriding

```
override fun onStimuliEnd() {
    val elapsed = Date().time - mTrialStartOnset

    if(elapsed > mTrialAbortTime - 10L)
        setAnswer(-1, elapsed)
    else
        setAnswer(elapsed.toInt(), elapsed)

    mStimuliHandler.removeCallbacksAndMessages(null)
    binding.root.removeView(mRespButton)
    super.onStimuliEnd()
}
```

It calculates the elapsed time from stimulus onset (set when it delivers the stimulus), if too much time has elapsed it set a results of -1, otherwise set the elapsed time. Make cleanups and call the super method

Temporal Bisection task

In a temporal bisection task, three stimuli are delivered: S1 at 0 ms, S3 at 1000 ms and S2 at varying latencies between 200 and 800 ms from S1. After S3, a dialog pops up and user must indicate whether S2 was closer to S1 or S3. S2 can be sent by an adaptive algorithm or one delivering it at fixed latencies. Stimuli can be an audio tone, a blinking image or a vibration. Trials can also be bimodal, introducing sensorial conflict by adding a second stimulus of different modality, around S2.
There are two main differences compared to Reaction times task, we must define the adaptive algorithm parameters, and, at the end of each trial, we must show the results dialog fragment.

1) Define custom trial class

More information is needed to implement a BIS trial. Specify the midlatency and other variables related to bimodal task

```
open class TrialBIS(id:Int=-1, type:Int, label:String, override var magnitude:Float, val
conflict_type:String, val duration:Long, val duration2:Long, val mid_latency:Long = 500L,
val conflict_magn:Float=0F, adoWrapper: ADOWrapper?=null, isTraining:Boolean=false):

TrialBasic(id, type, label, adoWrapper = adoWrapper, isTraining = isTraining)
```

2) Adaptive algorithm setup

In BIS task we will use two adaptive estimators. One sending stimuli after the midpoint (500 ms) and one before. This model allows properly managing the expected asymmetric temporal sensitivity of each participants.

```
private val nAdaptiveTrials             = 50
private val nAT_range_sub               = 300F
private val adoParams = ADOParams(guess_rate=0.5F, lapse_rate=0.04F, noise_perc=0.05F)

private lateinit var taskADAParams:TaskADAParams
private lateinit var adoWrapperPre:ADOWrapper
private lateinit var adoWrapperPost:ADOWrapper



override fun initTest() {

       adoWrapperPre   = ADOWrapper("adopywrapper.BISRelADOpyWrapper",
                                    "BISRelADOpyWrapper", adoParams, taskADAParams)

       adoWrapperPost  = ADOWrapper("adopywrapper.BISRelADOpyWrapper",
                                    "BISRelADOpyWrapper", adoParams, taskADAParams)


       if (subject.trman_type == TEST_TRMAN_FIXED)
           FixedTrialsManager(createUnimodalTrials())
       else
           AdaptiveTrialsManager(createTrialsAdaptive())

}
```

To create its trials, we define

```
private fun createTrialsAdaptive():List<TrialBasic>{
    val trials: MutableList<TrialBasic> = mutableListOf()

    val numFixedTrials = 10
    val numAdaptivePairs = (nAdaptiveTrials - numFixedTrials) / 2

    repeat(numAdaptivePairs) {
        trials.add(TrialBISRel(-1, subject.type, STIMULUS_TYPE_AUDIO,
                   TrialsManager.ADAPTIVE_VALUE, true,  CONFLICT_TYPE_NONE,
                   STIMULUS_DURATION_AUDIO, mid_latency = midLatency,
                    adoWrapper = adoWrapperPre))
      trials.add(TrialBISRel(-1, subject.type, STIMULUS_TYPE_AUDIO,
                    TrialsManager.ADAPTIVE_VALUE, false, CONFLICT_TYPE_NONE,
                    STIMULUS_DURATION_AUDIO, mid_latency = midLatency,
                    adoWrapper = adoWrapperPost))
    }

    trials.add(TrialBISRel(-1, subject.type, currStimulusLabel, 225F, true,
conflictType, currStimulusDuration, currStimulusDuration2))
    trials.add(TrialBISRel(-1, subject.type, currStimulusLabel, 175F, true,
conflictType, currStimulusDuration, currStimulusDuration2))
    trials.add(TrialBISRel(-1, subject.type, currStimulusLabel, 125F, true,
conflictType, currStimulusDuration, currStimulusDuration2))
    trials.add(TrialBISRel(-1, subject.type, currStimulusLabel, 75F , true,
conflictType, currStimulusDuration, currStimulusDuration2))
    trials.add(TrialBISRel(-1, subject.type, currStimulusLabel, 25F , true,
```

```
conflictType, currStimulusDuration, currStimulusDuration2))
    trials.add(TrialBISRel(-1, subject.type, currStimulusLabel, 25F , false,
conflictType, currStimulusDuration, currStimulusDuration2))
    trials.add(TrialBISRel(-1, subject.type, currStimulusLabel, 750F, false,
conflictType, currStimulusDuration, currStimulusDuration2))
    trials.add(TrialBISRel(-1, subject.type, currStimulusLabel, 125F, false,
conflictType, currStimulusDuration, currStimulusDuration2))
    trials.add(TrialBISRel(-1, subject.type, currStimulusLabel, 175F, false,
conflictType, currStimulusDuration, currStimulusDuration2))
    trials.add(TrialBISRel(-1, subject.type, currStimulusLabel, 225F, false,
conflictType, currStimulusDuration, currStimulusDuration2))

    trials.shuffle()
    return trials
}
```

3) End trial

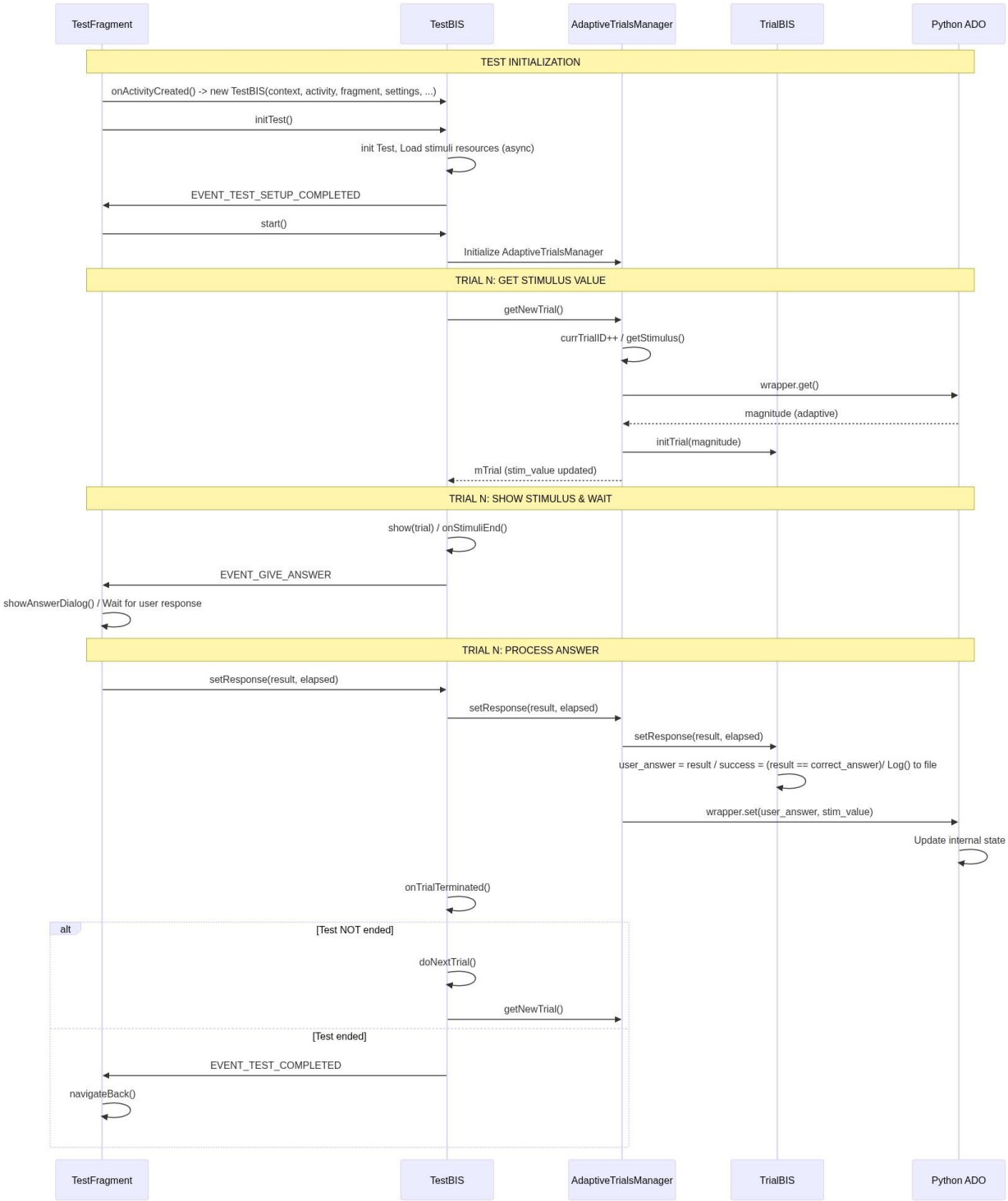
In `SettingsBIS`, nextTrailModality: Int = TestBasic.TEST_NEXTTRIAL_ANSWER

Consequently, trial end is managed by the super class `TestBasic`. The show function of TestBIS has only to define a last `mStimuliHandler.postDelayed` that calls `onStimuliEnd()` after S3 has been displayed.

From here on, data flow is managed within the framework, with `TestBasic` that emits a `testEvent.accept(Triple(EVENT_GIVE_ANSWER, null, listOf())),` that is observed in `TestFragment` which calls `showAnswerDialog(TRG_REQ_CODE_ANSWER),` that opens the dialog that let user indicate the correct answer.

Once the user responds and closes the dialog, a set of setResponse methods are called from TestFragment -> TestBis -> AdaptiveTrialsManager -> TrialBIS and finally updates the Adaptive model.

The interactions among these classes is summarized in the schema below.


TestFragment
TestBIS
AdaptiveTrialsManager
TrialBIS
Python ADO
TEST INITIALIZATION
onActivityCreated() -> new TestBIS(context, activity, fragment, settings, ...)
initTest()
init Test, Load stimuli resources (async)
EVENT_TEST_SETUP_COMPLETED
start()
Initialize AdaptiveTrialsManager
TRIAL N: GET STIMULUS VALUE
getNewTrial()
currTrialID++ / getStimulus()
wrapper.get()
magnitude (adaptive)
initTrial(magnitude)
mTrial (stim_value updated)
TRIAL N: SHOW STIMULUS & WAIT
show(trial) / onStimuliEnd()
EVENT_GIVE_ANSWER
showAnswerDialog() / Wait for user response
TRIAL N: PROCESS ANSWER
setResponse(result, elapsed)
setResponse(result, elapsed)
setResponse(result, elapsed)
user_answer = result / success = (result == correct_answer)/ Log() to file
wrapper.set(user_answer, stim_value)
Update internal state
onTrialTerminated()
alt
[Test NOT ended]
doNextTrial()
getNewTrial()
[Test ended]
EVENT_TEST_COMPLETED
navigateBack()
TestFragment
TestBIS
AdaptiveTrialsManager
TrialBIS
Python ADO